

# SQL

SQL (Structured Query Language) is a programming language which is used to manage data stored in relational databases like MySQL, MS Access, SQL Server, Oracle, Sybase, Informix, Postgres etc.

**SQL** is a database computer language designed for the retrieval and management of data in a relational databases like MySQL, MS Access, SQL Server, Oracle, Sybase, Informix, Postgres etc. **SQL** stands for **Structured Query Language**. SQL was developed in the 1970s by IBM Computer Scientists.

**SQL** is not a database management system, but it is a query language which is used to store and retrieve the data from a database or in simple words SQL is a language that communicates with databases.

## SQL Examples

Consider we have following CUSTOMERS table which stores customer's ID, Name, Age, Salary, City and Country

| ID | Name    | Age | Salary  | City       | Country |
|----|---------|-----|---------|------------|---------|
| 1  | Ramesh  | 32  | 2000.00 | Maryland   | USA     |
| 2  | Mukesh  | 40  | 5000.00 | New York   | USA     |
| 3  | Sumit   | 45  | 4500.00 | Muscat     | Oman    |
| 4  | Kaushik | 25  | 2500.00 | Kolkata    | India   |
| 5  | Hardik  | 29  | 3500.00 | Bhopal     | India   |
| 6  | Komal   | 38  | 3500.00 | Saharanpur | India   |
| 7  | Ayush   | 25  | 3500.00 | Delhi      | India   |

SQL makes it easy to manipulate this data using simple DML (Data Manipulation Language) Statements. For example, if we want to list down all the customers from USA then following will be the SQL query.

```
SELECT * FROM CUSTOMERS WHERE country = 'USA';
```

This will produce the following result:

| ID | Name   | Age | Salary  | City     | Country |
|----|--------|-----|---------|----------|---------|
| 1  | Ramesh | 32  | 2000.00 | Maryland | USA     |
| 2  | Mukesh | 40  | 5000.00 | New York | USA     |

## SQL Online Editor

We have provided **SQL Online Editor** which helps you to **Edit** and **Execute** the SQL code directly from your browser. Try to click the icon to run the following SQL code to be executed on **CUSTOMERS** table and print the records matching with the given condition.

```
SELECT * FROM CUSTOMERS WHERE country = 'USA';
```

So now, you do not need to do a sophisticated setup to execute all the examples given in this tutorial because we are providing you **Online SQL Editor**, which allows you to edit your code and compile it online. You can try our Online SQL Editor.

## SQL Basic Commands

We have a list of standard SQL commands to interact with relational databases, These commands are CREATE, SELECT, INSERT, UPDATE, DELETE, DROP and TRUNCATE and can be classified into the following groups based on their nature

### Data Definition Language (DDL)

A Data Definition Language (DDL) is a computer language which is used to create and modify the structure of database objects which include tables, views, schemas, and indexes etc.

| Command  | Description                                                                  | Demo |
|----------|------------------------------------------------------------------------------|------|
| CREATE   | Creates a new table, a view of a table, or other object in the database.     | Demo |
| ALTER    | Modifies an existing database object, such as a table.                       | Demo |
| DROP     | Deletes an entire table, a view of a table or other objects in the database. | Demo |
| TRUNCATE | Truncates the entire table in a go.                                          | Demo |

### Data Manipulation Language (DML)

A Data Manipulation Language (DML) is a computer programming language which is used for adding, deleting, and modifying data in a database.

| Command | Description                                        |
|---------|----------------------------------------------------|
| SELECT  | Retrieves certain records from one or more tables. |
| INSERT  | Creates a record.                                  |
| UPDATE  | Modifies records.                                  |
| DELETE  | Deletes records.                                   |

### Data Control Language (DCL)

Data Control Language (DCL) is a computer programming language which is used to control access to data stored in a database.

| Command | Description                              |
|---------|------------------------------------------|
| GRANT   | Gives a privilege to user                |
| REVOKE  | Takes back privileges granted from user. |

## Why to Learn SQL?

SQL (Structured Query Language) is a MUST for the students and working professionals to become a great Software Engineer specially when they are working in Software Development Domain. SQL is the most common language used almost in every application software including banking, finance, education, security etc. to store and manipulate data.

SQL is fairly easy to learn, so if you are starting to learn any programming language then it is very much advised that you should also learn SQL and other Database related concepts to become a complete Software Programmer. There are many good reasons which makes SQL as the first choice of any programmer

SQL is the standard language for any Relational Database System. All the Relational Database Management Systems (RDBMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their standard database language.

Also, software industry is using different dialects of SQL, such as

- MS SQL Server using T-SQL,
- Oracle using PL/SQL,
- MS Access version of SQL is called JET SQL (native format) etc.

## SQL Applications

SQL is one of the most widely used Query Language over the databases. SQL provides following functionality to the database programmers

- Execute different database queries against a database.
- Define the data in a database and manipulate that data.
- Create data in a relational database management system.
- Access data from the relational database management system.
- Create and drop databases and tables.
- Create and maintain database users.
- Create view, stored procedure, functions in a database.
- Set permissions on tables, procedures and views.

## Who Should Learn SQL

This **SQL tutorial** will help both students as well as working professionals who want to develop applications based on some databases like banking systems, support systems, information systems, web websites, mobile apps or personal blogs etc. We recommend reading this tutorial, in the sequence listed in the left side menu.

Today, SQL is an essential language to learn for anyone involved in the software application development process including Software Developers, Software Designers, and Project Managers etc.

## **Prerequisites to Learn SQL**

Though we have tried our best to present the SQL concepts in a simple and easy way, still before you start learning SQL concepts given in this tutorial, it is assumed that you are already aware about some basic concepts of computer science, what is a database, especially the basics of RDBMS and associated concepts.

This tutorial will give you enough understanding on the various concepts of SQL along with suitable examples so that you can start your Software Development journey immediately after finishing this tutorial.

developed a great learning material for SQL which will help you prepare for the technical interviews and certification exams based on SQL. So, start learning SQL using our simple and effective tutorial anywhere and anytime absolutely at your pace.

# Roadmap

The **SQL Roadmap** is like a simple study plan for learning SQL. It shows you which topics to learn first and what to learn next, starting from basic ideas and moving to advanced skills. This helps you understand how to work with databases step by step.

It is helpful for everyone i.e. students, developers, data analysts, or anyone who wants to manage databases because it makes sure you learn all the important concepts in the right order.

## 1. SQL Basics

SQL stands for **Structured Query Language**. It is the standard language used to store, retrieve, update, and delete data in relational databases. In other words, whenever you need to work with data inside a database, SQL provides the commands to do it.

### 1.1 RDBMS Concepts

An **RDBMS** (Relational Database Management System) is software that stores information in **tables**, which look like spreadsheets with rows and columns. Each table usually represents one type of real-world thing, such as employees, products, or orders.

- **Relation (Table):** A table is called a **relation** and represents a specific entity.  
Example: an **employees** table stores details like name, email, and salary.
- **Primary Key:** A special column (or set of columns) that **uniquely identifies each row** in a table.

Example: an **id** column that gives each employee a unique number.

- **Foreign Key:** A column in one table that **links to the primary key of another table**, creating a relationship between them.

Example: an **orders** table might have an **employee\_id** column to show which employee handled the order.

- **Normalization:** The process of organizing the database so that **data is not unnecessarily repeated**.

Example: instead of storing a customer's address in every order, keep it in a separate **customers** table and link it using a foreign key.

### 1.2 RDBMS Benefits

Relational databases provide several advantages:

- **Data integrity:** Ensures that data stays accurate and consistent.
- **ACID transactions:** Guarantees reliable operations so data is never left incomplete.
- **Concurrent access:** Allows multiple users to safely work with data at the same time.
- **Powerful queries:** Enables complex searching and combining of data using SQL.

- **Mature tools:** RDBMS technology is stable and well-supported after decades of use.

## 1.3 Database-level commands

SQL provides commands to create and delete entire databases:

- **CREATE:** Makes a new database. Example:

```
CREATE DATABASE company;
```

- **DROP:** Permanently deletes a database (this cannot be undone). Example:

```
DROP DATABASE company;
```

## 2. SQL Basic Syntax

SQL statements follow a specific structure and let you perform key operations such as **selecting** data, adding new records, updating existing information, and deleting rows when needed. Understanding the basic syntax is the first step to writing SQL queries.

### 2.1 SQL Comments

Comments are notes inside your SQL code that help to explain what the code does. They are ignored by the database engine and are only for humans to read.

```
-- Single-line comment
```

```
/* Multi-line  
comment */
```

### 2.2 SQL Data Types

Each column in a database table has a **data type** that defines what kind of values it can store.

- **INT:** Whole numbers (integers) such as 1, 42, or -10.
- **VARCHAR(n):** Text of variable length, up to **n** characters. Example: names or email addresses.
- **DECIMAL(p,s):** Numbers with a fixed precision. **p** is total digits and **s** is digits after the decimal point. Example: DECIMAL(6,2) can store 1234.56.
- **DATE / TIMESTAMP:** Dates and times. DATE stores only the date, while TIMESTAMP stores both date and time.

### 2.3 SQL Operators & Expressions

You can use **operators** (arithmetic, comparison, and logical) in SQL to create expressions and filter data in queries. This is commonly done in the **WHERE** clause.

```
SELECT id, salary, salary * 0.1 AS bonus  
FROM employees  
WHERE salary > 50000 AND department = 'Sales';
```

In this example:

- The query retrieves each employee's **id**, **salary**, and calculates a **bonus** (10% of salary).
- The **WHERE** clause filters only employees earning more than 50,000 and working in the Sales department.

## 2.4 Core DML Statements

SQL's core Data Manipulation Language (DML) commands allow you to manage the data inside tables:

- **SELECT:** Retrieve (read) data from a table.
- **INSERT:** Add new rows of data.
- **UPDATE:** Modify existing rows.
- **DELETE:** Remove rows from a table.

```
-- Insert a new employee
INSERT INTO employees (name, department, salary)
VALUES ('Alice', 'Sales', 65000);
```

```
-- Update salary for the employee with id = 5
UPDATE employees
SET salary = salary + 2000
WHERE id = 5;
```

```
-- Delete the employee with id = 10
DELETE FROM employees WHERE id = 10;
```

## 3. Data Definition Language (DDL)

DDL commands define the structure of the database itself. They are used to **create**, **change**, or **remove** database objects like tables, constraints, and indexes. Think of DDL as the "blueprint" stage deciding how data will be organized.

### 3.1 CREATE / ALTER / DROP

These commands are the core tools to design and modify your database schema:

- **CREATE:** Build new database objects such as tables.
- **ALTER:** Change an existing table, for example by adding a column.
- **DROP:** Permanently delete a database object such as a table.

```
CREATE TABLE employees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  department_id INT,
  salary DECIMAL(10,2),
  hire_date DATE
);
```

```
ALTER TABLE employees ADD COLUMN email VARCHAR(255);
```

```
DROP TABLE old_table;
```

Explanation:

- The first statement creates a table named **employees** with columns for id, name, department, salary, and hire date.  
The **PRIMARY KEY** and **AUTO\_INCREMENT** ensure each employee has a unique id that increases automatically.
- The second statement adds a new column **email** to the existing **employees** table.
- The third statement permanently deletes the table named **old\_table**.

### 3.2 TRUNCATE vs DELETE

Both commands remove data, but work differently:

- **TRUNCATE:** Quickly removes **all rows** from a table. It resets the storage space and uses minimal logging.

It is fast but cannot remove only some rows.

- **DELETE:** Removes rows selectively using a **WHERE** clause.

Each row removal is logged, so it can be rolled back inside a transaction if needed.

### 3.3 TEMPORARY & CLONE

- **TEMPORARY tables:** Created for the current session only. They disappear automatically when the session ends.

Example: **CREATE TEMPORARY TABLE tmp(...);**

- **CLONE:** Some database systems provide commands to duplicate the structure and/or data of an existing table.

The exact syntax depends on the database engine.

### 3.4 Constraints

Constraints are rules that ensure the accuracy and consistency of data. They enforce certain conditions when inserting or updating records:

- **PRIMARY KEY:** Ensures each row has a unique identifier.
- **FOREIGN KEY:** Maintains a valid link between tables to preserve referential integrity.
- **UNIQUE:** Guarantees that no two rows have the same value in this column.
- **NOT NULL:** Requires that a column always has a value.
- **CHECK:** Ensures a column's value meets a specified condition (e.g., salary > 0).

```
CREATE TABLE orders (
  id INT PRIMARY KEY,
  customer_id INT,
  amount DECIMAL(10,2) CHECK (amount > 0),
  CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

In this example:

- The **CHECK** constraint makes sure the **amount** is greater than zero.



- The **FOREIGN KEY** constraint named **fk\_customer** ensures that each **customer\_id** matches an existing **id** in the **customers** table.

## 4. Data Manipulation Language (DML)

DML commands are used to **retrieve** and **manipulate** the data stored in database tables. This section focuses on selecting data and combining data from multiple tables using joins.

### 4.1 SELECT basics

The basic pattern for reading data is:

#### **SELECT ... FROM ... WHERE ...**

```
SELECT e.name, d.name AS department, e.salary
FROM employees e
JOIN departments d ON e.department_id = d.id
WHERE e.salary > 50000
ORDER BY e.salary DESC
LIMIT 10;
```

Explanation:

- Retrieves **employee names, department names, and salaries**.
- **JOIN** combines employee and department tables based on **department\_id**.
- **WHERE** filters employees earning more than 50,000.
- **ORDER BY** sorts salaries in descending order.
- **LIMIT** returns only the top 10 results.

### 4.2 JOIN types

Joins combine data from multiple tables. The common types are:

- **INNER JOIN:** Returns only rows with matches in both tables.
- **LEFT JOIN:** Returns all rows from the left table, with matching rows from the right table if available.
- **RIGHT JOIN:** Returns all rows from the right table, with matching rows from the left table (less common).
- **FULL JOIN:** Returns all rows where a match exists in either table (supported in some DBs).
- **CROSS JOIN:** Cartesian product of both tables; use carefully.

```
-- Example: LEFT JOIN
SELECT c.name, o.id AS order_id
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id;
```

### 4.3 Aggregate Functions & GROUPING

Aggregates summarize multiple rows into single results. Use **GROUP BY** to group rows and optionally **HAVING** to filter groups.

- **SUM:** Total of a numeric column.

- **COUNT:** Number of rows (or non-NULL values).
- **AVG:** Average value of a column.
- **MIN:** Minimum value.
- **MAX:** Maximum value.

```
SELECT department_id,
       COUNT(*) AS staff_count,
       AVG(salary) AS avg_salary,
       SUM(salary) AS total_pay
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

## 4.4 Indexes

Indexes improve query performance by providing fast lookup paths.

- **Create Index:** **CREATE INDEX idx\_name ON table(col);**
- **Drop Index:** **DROP INDEX idx\_name;** (varies by database)
- **Unique Index:** Ensures column values are unique.
- **Clustered vs Non-Clustered:** Clustered changes physical order of rows, non-clustered is separate.

```
CREATE INDEX idx_employees_department ON employees(department_id);
CREATE UNIQUE INDEX idx_customers_email ON customers(email);
```

# Overview

## What is SQL?

SQL (Structured Query Language) is a language to operate databases; it includes Database Creation, Database Deletion, Fetching Data Rows, Modifying & Deleting Data rows, etc.

**SQL** stands for **Structured Query Language** which is a computer language for storing, manipulating and retrieving data stored in a relational database. SQL was developed in the 1970s by IBM Computer Scientists and became a standard of the American National Standards Institute (ANSI) in 1986, and the International Organization for Standardization (ISO) in 1987.

Though SQL is an **ANSI** (American National Standards Institute) standard language, but there are many different dialects of the SQL language like MS SQL Server is using T-SQL and Oracle is using PL/SQL.

SQL is the standard language to communicate with Relational Database Systems. All the Relational Database Management Systems (RDMS) like MySQL, MS Access, Oracle, Sybase, Informix, Postgres and SQL Server use SQL as their Standard Database Language.

## Why SQL?

SQL is widely popular because it offers the following advantages

- Allows users to access data in the relational database management systems.
- Allows users to describe the data.
- Allows users to define the data in a database and manipulate that data.
- Allows to embed within other languages using SQL modules, libraries & pre-compilers.
- Allows users to create and drop databases and tables.
- Allows users to create view, stored procedure, functions in a database.
- Allows users to set permissions on tables, procedures and views.

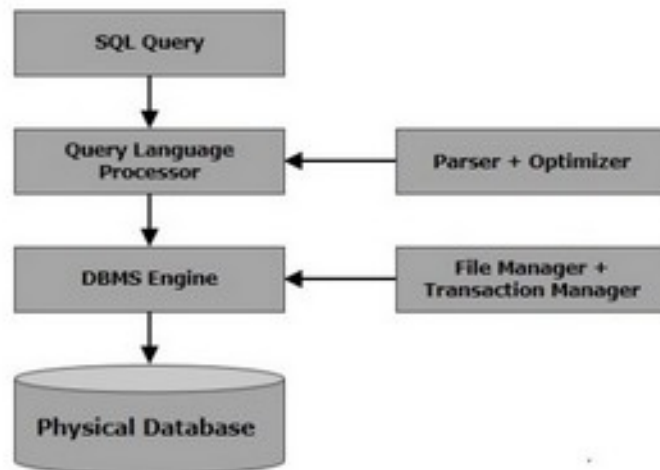
## How SQL Works?

When you are executing an SQL command for any RDBMS, the system determines the best way to carry out your request and SQL engine figures out how to interpret the task.

There are various components included in this process. These components are

- Query Dispatcher
- Optimization Engines
- Classic Query Engine
- SQL Query Engine, etc.

A classic query engine handles all the non-SQL queries, but a SQL query engine won't handle logical files. Following is a simple diagram showing the SQL Architecture



# RDBMS Concepts

## What is RDBMS?

RDBMS stands for **R**elational **D**atabase **M**anagement **S**ystem. RDBMS is the basis for SQL, and for all modern database systems like MS SQL Server, IBM DB2, Oracle, MySQL, and Microsoft Access.

A Relational database management system (RDBMS) is a database management system (DBMS) that is based on the relational model as introduced by E. F. Codd in 1970.

## What is a Table?

The data in an RDBMS is stored in database objects known as **tables**. This table is basically a collection of related data entries and it consists of numerous columns and rows.

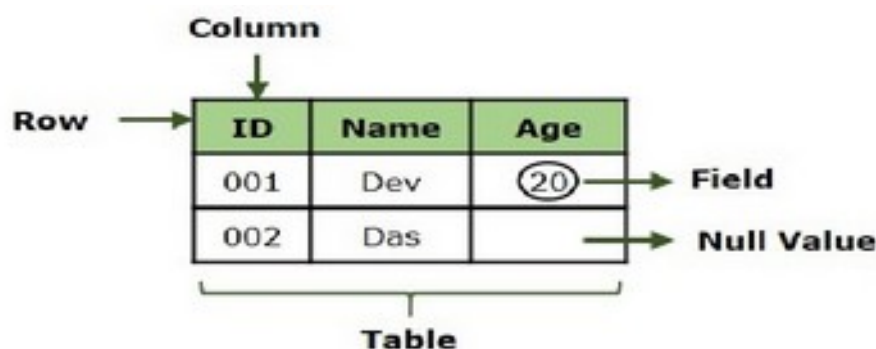
Remember, a table is the most common and simplest form of data storage in a relational database. Following is an example of a CUSTOMERS table which stores customer's ID, Name, Age, Salary, City and Country

| ID | Name    | Age | Salary  | City       | Country |
|----|---------|-----|---------|------------|---------|
| 1  | Ramesh  | 32  | 2000.00 | Hyderabad  | India   |
| 2  | Mukesh  | 40  | 5000.00 | New York   | USA     |
| 3  | Sumit   | 45  | 4500.00 | Muscat     | Oman    |
| 4  | Kaushik | 25  | 2500.00 | Kolkata    | India   |
| 5  | Hardik  | 29  | 3500.00 | Bhopal     | India   |
| 6  | Komal   | 38  | 3500.00 | Saharanpur | India   |
| 7  | Ayush   | 25  | 3500.00 | Delhi      | India   |
| 8  | Javed   | 29  | 3700.00 | Delhi      | India   |

## What is a Field?

Every table is broken up into smaller entities called fields. A field is a column in a table that is designed to maintain specific information about every record in the table.

For example, our CUSTOMERS table consists of different fields like ID, Name, Age, Salary, City and Country.



## What is a Record or a Row?

A record is also called as a row of data is each individual entry that exists in a table. For example, there are 7 records in the above CUSTOMERS table. Following is a single row of data or record in the CUSTOMERS table

| ID | Name   | Age | Salary  | City      | Country |
|----|--------|-----|---------|-----------|---------|
| 1  | Ramesh | 32  | 2000.00 | Hyderabad | India   |

A record is a horizontal entity in a table.

## What is a Column?

A column is a vertical entity in a table that contains all information associated with a specific field in a table.

For example, our CUSTOMERS table have different columns to represent ID, Name, Age, Salary, City and Country.

## What is a NULL Value?

A NULL value in a table is a value in a field that appears to be blank, which means a field with a NULL value is a field with no value.

It is very important to understand that a NULL value is different than a zero value or a field that contains spaces. A field with a NULL value is the one that has been left blank during a record creation. Following table has three records where first record has NULL value for the salary and second record has a zero value for the salary.

| ID | Name   | Age | Salary  | City      | Country |
|----|--------|-----|---------|-----------|---------|
| 1  | Ramesh | 32  |         | Hyderabad | India   |
| 2  | Mukesh | 40  | 00.00   | New York  | USA     |
| 3  | Sumit  | 45  | 4500.00 | Muscat    | Oman    |

## SQL Constraints

Constraints are the rules enforced on data columns on a table. These are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the database.

Constraints can either be column level or table level. Column level constraints are applied only to one column whereas, table level constraints are applied to the entire table.

Following are some of the most commonly used constraints available in SQL

| S.No. | Constraints |
|-------|-------------|
|-------|-------------|

|  |                            |
|--|----------------------------|
|  | <b>NOT NULL Constraint</b> |
|--|----------------------------|

|   |                                                 |
|---|-------------------------------------------------|
| 1 | Ensures that a column cannot have a NULL value. |
|---|-------------------------------------------------|

|   |                           |
|---|---------------------------|
| 2 | <b>DEFAULT Constraint</b> |
|---|---------------------------|

Provides a default value for a column when none is specified.

### **UNIQUE Key**

- 3 Ensures that all the values in a column are different.

### **PRIMARY Key**

- 4 Uniquely identifies each row/record in a database table.

### **FOREIGN Key**

- 5 Uniquely identifies a row/record in any another database table.

### **CHECK Constraint**

- 6 Ensures that all values in a column satisfy certain conditions.

### **INDEX Constraint**

- 7 Used to create and retrieve data from the database very quickly.

## **Data Integrity**

The following categories of data integrity exist with each RDBMS

- **Entity Integrity** This ensures that there are no duplicate rows in a table.
- **Domain Integrity** Enforces valid entries for a given column by restricting the type, the format, or the range of values.
- **Referential integrity** Rows cannot be deleted, which are used by other records.
- **User-Defined Integrity** Enforces some specific business rules that do not fall into entity, domain or referential integrity.

## **Database Normalization**

Database normalization is the process of efficiently organizing data in a database. There are two reasons of this normalization process

- Eliminating redundant data, for example, storing the same data in more than one table.
- Ensuring data dependencies make sense.

Both these reasons are worthy goals as they reduce the amount of space a database consumes and ensures that data is logically stored. Normalization consists of a series of guidelines that help guide you in creating a good database structure.

Normalization guidelines are divided into normal forms; think of a form as the format or the way a database structure is laid out. The aim of normal forms is to organize the database structure, so that it complies with the rules of first normal form, then second normal form and finally the third normal form.